VOLUME II · DRILL PACK · MOCK PAPER · WALK-IN CHEATSHEET

CMS 702 — Practice, Answer Skeletons & Final Recall

The companion to the Volume I study guide. Volume I tells you *what* to know; this one makes you *produce* it under exam conditions — a full mock paper with model answers, worked drills on every calculable topic, and a one-page sheet to scan at the door.

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Monday 03 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer **Units:** 3

HOW TO USE THIS PACK TONIGHT

TIME YOU HAVE	DO THIS
3 hours	Sit the mock paper (§2) closed-book for 2 hours → mark yourself against §3 → drill only the sections you lost marks in (§4–§7) → read §9 traps → sleep.
90 minutes	Answer skeletons (§1) → mock paper Q1 and Q3 only → the BST and hashing drills (§4, §6) → §9 traps.
45 minutes	Answer skeletons (§1) → rapid-fire recall (§8), covering the answer column → §9 traps → §10 walk-in sheet.
10 minutes at the door	§10 only. Nothing else.

Rule for tonight: do not re-read Volume I passively. Reading feels productive and teaches nothing at this stage. Write the answers out by hand — the exam is a writing test, not a reading test.

1 Answer skeletons — the shape that earns marks

the lecturer's paper is bookwork, so marks are awarded for **structure and coverage**, not for insight. Every question you meet is one of five archetypes. Memorise the skeleton, then pour the content in.

IF THE QUESTION SAYS…	WRITE THIS SKELETON	WORKED OPENING LINE
"**Define X**" or "What is X?"	**1.** One-sentence formal definition · **2.** One sentence expanding it (what it is *for*, or how it works) · **3.** A numbered list of its types/components/properties · **4.** One concrete example · **5.** A small diagram if the term has a standard picture	"A **stack** is an abstract data type that implements the Last-In-First-Out (LIFO) principle. It is open at one end only, where insertion is called PUSH and removal is called POP…"
"**Define X, with examples**"	The same, but the example is **not optional** — give **two or three**, each with one line saying how it works. A table of examples beats a paragraph.	"…Examples of sort algorithms include merge sort, which divides the list in half, sorts each half and merges them; quick sort, which…"
"**Differentiate between X and Y**"	**1.** Define both terms first (2 sentences) · **2.** A table with a **Basis of comparison** column and one row per basis — aim for 6–10 rows · **3.** A one-sentence summary underneath	"A stack is open at one end and follows LIFO; a queue is open at both ends and follows FIFO. The differences are tabulated below."
"**State / list / explain the importance of X**"	**1.** Define X in one line · **2. Numbered** points, 8–9 of them, each a **bold lead phrase** followed by one explaining sentence · **3.** A closing supporting list (types, operations)	"**Efficient access and retrieval of data.** The right data structure lets a program find an item quickly — O(n) in an unsorted list, O(log n)…"
"**…with a diagram**" / "**illustrate**"	**Draw first, write second.** Label both axes or all nodes, name the diagram, and refer to it in the prose ("as shown in the figure above"). An unlabelled diagram scores almost nothing.	Axes labelled *running time* and *input size n*, curves labelled `f(n)` and `c·g(n)`, and n_0 marked on the horizontal axis.

FOUR HABITS THAT QUIETLY COST MARKS

1. Starting to write before deciding the skeleton — the answer wanders and repeats. **2.** Writing one long paragraph where a table was asked for. **3.** Stopping at three points on an open-ended question because you ran out of *ideas* rather than *time* — the ninth point scores exactly as much as the first. **4.** Leaving the diagram until the end and running out of time; it is the cheapest mark on the paper.

2 Mock paper — sit this closed-book

RIVERS STATE UNIVERSITY · PGD COMPUTER SCIENCE · CMS 702 — DATA STRUCTURES AND COMPUTER ALGORITHMS

Mock examination · Time allowed: 3 hours · **Answer ALL questions** · Each question carries 20 marks · Diagrams must be labelled

Q	QUESTION	MARKS
1	**(a)** Define asymptotic analysis and explain why it is used in the analysis of algorithms. **(b)** State and explain the three cases under which the running time of an algorithm is analysed. **(c)** State the three asymptotic notations, give the formal definition of each, and illustrate all three with labelled diagrams.	4 + 6 + 10
2	**(a)** Define a data structure and state the three categories of data types, with two examples of each. **(b)** State and briefly explain the six basic operations performed on data structures. **(c)** With the aid of a diagram, explain how a node is deleted from a singly linked list.	6 + 6 + 8
3	**(a)** Define hashing, a hash function and a hash table. **(b)** Using the strings `{"be", "cab", "fade"}` with `a = 1, b = 2 … z = 26`, compute the hash index of each string in a table of size 11 and show the resulting hash table. **(c)** What is a collision? State and explain two methods of handling collisions. **(d)** Define the load factor and compute it for a table of size 13 holding 9 items.	6 + 6 + 5 + 3
4	**(a)** Define a binary search tree and state its properties. **(b)** Construct a BST from the values **50, 30, 70, 20, 40, 60, 80, 35**, inserted in that order. **(c)** Give the in-order, pre-order and post-order traversals of your tree. **(d)** State the depth and height of nodes 50, 30 and 35, and find the in-order successor of 40 and the in-order predecessor of 50.	5 + 5 + 6 + 4
5	**(a)** Differentiate between a stack and a queue, using a table. **(b)** Define a sort algorithm and a search algorithm, giving two examples of each with their time complexities. **(c)** Differentiate between sequential search and binary search, and state the condition binary search requires.	8 + 8 + 4

3 Model answers & mark allocation

Question 1 — asymptotic analysis

(a) Asymptotic analysis refers to computing the running time of any piece of code or operation in a mathematical unit of computation, expressed in terms of a function `f(n)` where `n` is the size of the input. It is also described as the method of describing the **limiting behaviour** of an algorithm — how it behaves as the input grows towards infinity. **It is used because** the actual running time depends on the hardware, the compiler and the programming language; by ignoring machine-dependent constants and concentrating on the **rate of growth**, we obtain a measure of efficiency that holds on any machine. [1 mark definition · 1 mark f(n) · 1 mark limiting behaviour · 1 mark machine independence]

(b) Worst case — the maximum time required by the algorithm; the case most commonly used, because it guarantees the algorithm will never take longer. **Best case** — the minimum time the algorithm will take to perform its complete execution. **Average case** — the average time to complete execution, assuming all inputs of a given size follow a certain distribution. [2 marks each]

(c) Big O (O) — the **upper** bound of the growth rate; measures the worst case; the algorithm will never be slower. **Theta (Θ)** — the **tight** bound, expressing both the upper and the lower bound; the most precise notation. **Omega (Ω)** — the **lower** bound, expressing only the best case; the algorithm will never be faster.

$$f(n) = O(g(n)) \iff 0 \le f(n) \le c \cdot g(n) \quad \cdot \quad f(n) = \Theta(g(n)) \iff 0 \le k_1 \cdot g(n) \le f(n) \le k_2 \cdot g(n) \quad \cdot$$
$$f(n) = \Omega(g(n)) \iff 0 \le c \cdot g(n) \le f(n) \quad (\text{all for } n \ge n_0)$$

Then the three sketches from Volume I §3: **O** — f(n) below c·g(n); **Θ** — f(n) trapped between $k_1 \cdot g(n)$ and $k_2 \cdot g(n)$; **Ω** — f(n) above c·g(n). Axes labelled *running time* and *input size n*; n_0 marked on each. [2 marks per notation explained · 4 marks for three labelled diagrams]

Question 2 — data structures

(a) A **data structure** is a data organization, management and storage format that enables efficient access and modification; it is a collection of data values, the relationships among them, and the operations applicable to them. Categories: **inbuilt/primitive** — integer, float, boolean · **derived** — stack, queue, list, array · **complex** — linked list, tree, graph.

(b) Traversal — visiting every element once · **Searching** — locating a given element · **Sorting** — arranging elements in a specific order · **Merging** — combining two structures into one · **Insertion** — adding a new element · **Deletion** — removing an existing element.

(c) Locate the target node, then redirect the previous node's pointer past it to the target's next node, so the target is bypassed and can be freed:

```
Before:  Head → [A|•] → [TARGET|•] → [C|•] → NULL
After:   Head → [A|•] ──────────────→ [C|•] → NULL
```

Say explicitly: **no elements are shifted** — this is why deletion in a linked list is O(1) once the node is found, against O(n) in an array. [Diagram 4 marks · explanation 4 marks]

Question 3 — hashing (fully worked)

(a) Hashing means lookup — the most widely used technique to find aggregate data by key or id; it maps a large set of arbitrary data to a tabular index using a hash function, allowing lookup, update and retrieval in **constant time O(1)**. A **hash function** receives the input key and returns the index of an element in an array called the hash table. A **hash table** maps keys to values using that function, storing data in an associative manner in an array where each value has its own unique index.

(b) Letter values `a = 1 … z = 26`, table size 11, rule `index = sum mod 11`:

```
be   = b + e         = 2 + 5         =  7   →   7 mod 11 = index 7
cab  = c + a + b     = 3 + 1 + 2     =  6   →   6 mod 11 = index 6
fade = f + a + d + e = 6 + 1 + 4 + 5 = 16   →  16 mod 11 = index 5

index:   0    1    2    3    4     5     6     7    8    9   10
       ┌────┬────┬────┬────┬────┬─────┬─────┬─────┬────┬────┬────┐
       │    │    │    │    │    │ fade │ cab │ be  │    │    │    │
       └────┴────┴────┴────┴────┴─────┴─────┴─────┴────┴────┴────┘
```

Always show all three steps — the letter sum, the modulo, the slot. Marks are given per step, not per final answer.

(c) A **collision** occurs when `h(x) = h(y)` — two different keys map to the same hash value. **Separate chaining** — each cell of the hash table points to a linked list of records, and colliding keys are appended to that list. **Open addressing** — all elements are stored in the hash table itself, and a colliding key is placed in the next free slot found by a probing sequence.

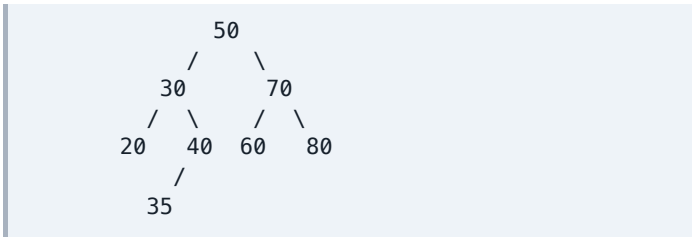
(d) Load factor = number of items the hash table contains ÷ size of the hash table = **9 / 13 = 0.69** (2 d.p.). A high load factor means more collisions and slower lookup, which is why a low load factor is a property of a good hash function.

Question 4 — BST (fully worked)

(a) A **BST** is a node-based binary tree in which the value of the left node is less than its parent and the value of the right node is greater than its parent. Properties: left sub-tree strictly less · right sub-tree strictly greater · the rule applies recursively to every sub-tree · no duplicate keys.

(b) Inserting **50, 30, 70, 20, 40, 60, 80, 35** — start at the root each time, go left if smaller, right if larger:

```
50 → root
30 < 50 → left of 50
70 > 50 → right of 50
20 < 50, < 30 → left of 30
40 < 50, > 30 → right of 30
60 > 50, < 70 → left of 70
80 > 50, > 70 → right of 70
35 < 50, > 30, < 40 → left of 40
```



(c) In-order (L-Root-R): `20, 30, 35, 40, 50, 60, 70, 80` — sorted, which confirms the tree is correct. **Pre-order** (Root-L-R): `50, 30, 20, 40, 35, 70, 60, 80`. **Post-order** (L-R-Root): `20, 35, 40, 30, 60, 80, 70, 50`.

(d) 50: depth 0, height 3 (50 → 30 → 40 → 35 is the longest downward path). **30:** depth 1, height 2. **35:** depth 3, height 0 — it is a leaf. **In-order successor of 40** = the next value in the in-order sequence = **50**. **In-order predecessor of 50** = the previous value = **40**.

Question 5 — stacks, queues, sorting and searching

(a) Define both, then the table: LIFO vs FIFO · one end vs both ends · PUSH/POP vs Enqueue()/Dequeue() · one pointer (top) vs two (front, rear) · pile of plates vs queue of people · recursion, undo and DFS vs CPU scheduling, printer spooling and BFS.

(b) A **sort algorithm** arranges the elements of a list in a specific order — ascending, descending or lexicographic. Examples: **merge sort**, which divides the list in half, sorts each half and merges them, O(n log n); **quick sort**, which partitions around a pivot and recurses, O(n log n) average and O(n^2) worst. A **search algorithm** finds a specific target within a dataset, enabling effective retrieval of information. Examples: **sequential search**, checking each element in turn, O(n); **binary search**, repeatedly halving a sorted list, O(log n).

(c) Sequential search needs no ordering and costs O(n) in the worst case; binary search compares with the middle element and discards half the list each time, costing O(log n), **but it requires the dataset to be sorted**.

4 BST drill set — with full solutions

DRILL 1 — TRAVERSE THIS TREE

Give the in-order, pre-order and post-order traversals, the height of the tree, and the degree of node B.

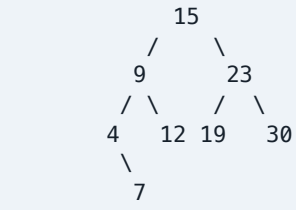
Solution. In-order `D, B, H, E, A, F, C, G` · Pre-order `A, B, D, E, H, C, F, G` · Post-order `D, H, E, B, F, G, C, A`. Height of the tree = **3** (A → B → E → H). Degree of B = **2**.

Note this is *not* a BST, so the in-order traversal is not sorted — the sorted-check only applies to binary *search* trees.

DRILL 2 — BUILD AND INTERROGATE

Insert **15, 9, 23, 4, 12, 19, 30, 7** into an empty BST, then give the three traversals, the height of the tree, the in-order successor of 12 and the in-order predecessor of 19.

Solution.



In-order `4, 7, 9, 12, 15, 19, 23, 30` · Pre-order `15, 9, 4, 7, 12, 23, 19, 30` · Post-order `7, 4, 12, 9, 19, 30, 23, 15`. Height = **3** (15 → 9 → 4 → 7). Successor of 12 = **15**; predecessor of 19 = **15**.

DRILL 3 — SUCCESSOR AND PREDECESSOR SWEEP

Using the Drill 2 tree, complete the table. Method: write the in-order traversal once, then read off neighbours — never trace pointers under time pressure.

NODE K	4	7	9	12	15	19	23	30
In-order successor	7	9	12	15	19	23	30	**−1**
In-order predecessor	**null**	4	7	9	12	15	19	23

The two end cases are the marks people drop: **−1** when no successor exists (largest node), **null** when no predecessor exists (smallest node).

Tree reconstruction drills

DRILL 4 — PRE-ORDER + IN-ORDER

```
Pre-order = G, D, A, F, E, M, H, Z
In-order  = A, D, E, F, G, H, M, Z
```

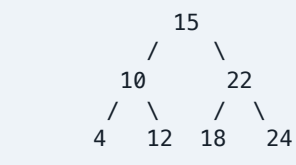
Solution. First of the pre-order is the root → **G**. In the in-order, `A D E F` lies left of G and `H M Z` right of it.
Left: pre `D A F E` → root **D**; in-order left of D is `A`, right is `E F` → F is the next root there, with E as its left child.
Right: pre `M H Z` → root **M**, with H left and Z right.

Check: in-order of the drawn tree = `A, D, E, F, G, H, M, Z` ✓ (and it is sorted, so this is also a valid BST). Post-order = `A, E, F, D, H, Z, M, G`.

DRILL 5 — POST-ORDER + IN-ORDER

```
Post-order = 4, 12, 10, 18, 24, 22, 15
In-order   = 4, 10, 12, 15, 18, 22, 24
```

Solution. Last of the post-order is the root → **15**. In the in-order, `4 10 12` is the left sub-tree and `18 22 24` the right.
Left: post `4 12 10` → root **10**, with 4 left and 12 right.
Right: post `18 24 22` → root **22**, with 18 left and 24 right.



Check: in-order = `4, 10, 12, 15, 18, 22, 24` ✓.
Pre-order = `15, 10, 4, 12, 22, 18, 24`.

The whole method in one line: pre-order gives the root at the **front**, post-order gives it at the **back**; the in-order always tells you

where to **split**. Recurse on each half.

Drill 6 — height, depth and degree

Give the depth and height of P, Q, T, V and Y; the height of the tree; the degree of R; and list all leaves.

Solution.

NODE	DEPTH	HEIGHT
P	0	4
Q	1	2
T	2	1
V	3	0
Y	4	0

Height of the tree = 4 (P → R → U → X → Y). **Degree of R** = 1 — it has only a right child. **Leaves:** V, W, Y.

Depth counts edges downward *from the root*; height counts edges upward *from the deepest leaf*. Every leaf has height 0.

5 Complexity drills — read the code, state the class

#	CODE OR SITUATION	ANSWER	WHY
1	`for i = 1 to n: print(i)`	**O(n)**	One loop over n elements — work proportional to input size.
2	`for i = 1 to n: for j = 1 to n: x++`	**O(n²)**	The inner loop runs n times for each of the n outer iterations → n × n operations.
3	`for i = 1 to n: …` then separately `for j = 1 to n: …`	**O(n)**	Sequential loops **add** — n + n = 2n, and constants are dropped. Only *nested* loops multiply.
4	`while n > 1: n = n / 2`	**O(log n)**	The problem size halves each step — the defining shape of a logarithmic algorithm.
5	`for i = 1 to n: for j = 1 to i: x++`	**O(n²)**	1 + 2 + … + n = n(n+1)/2, which is still quadratic once constants are dropped.
6	Accessing `A[i]` in an array	**O(1)**	The address is computed directly from the index — the time does not depend on the array's size.
7	Naïve recursive `fib(n)`	**O(2ⁿ)**	Each call spawns two more, and the same sub-problems are recomputed. Dynamic programming reduces it to O(n).
8	Binary search on a sorted array of **1024** items	**10 steps**	$\log_2 1024 = 10$ — at most ten comparisons, against up to 1024 for a sequential search.
9	Searching an unsorted list of 1 000 items, worst case	**1 000**	Sequential search must check every element when the target is last or absent — O(n).
10	Merge sort on any input	**O(n log n)**	log n levels of splitting, each level doing O(n) work to merge. Guaranteed — best, average and worst.
11	Quick sort on an already-sorted array with a first-element pivot	**O(n²)**	The pivot is always the smallest element, so each partition removes only one element — the worst case.
12	Lookup in a hash table with a good hash function	**O(1)**	The key is transformed straight into an index; only collisions degrade it.

6 Hashing drills — with the arithmetic shown

DRILL A — TABLE SIZE 7

Hash `{"hi", "if", "ad", "bc"}` with `a = 1 … z = 26` into a table of size 7.

```
hi = 8 + 9  = 17  → 17 mod 7 = index 3
if = 9 + 6  = 15  → 15 mod 7 = index 1
ad = 1 + 4  =  5  →  5 mod 7 = index 5
bc = 2 + 3  =  5  →  5 mod 7 = index 5  ← COLLISION
```

"ad" and "bc" collide — different keys, same hash value, i.e. `h(x) = h(y)`. Resolve by **separate chaining** (slot 5 points to a linked list holding both) or **open addressing** (place "bc" in the next free slot, index 6).

DRILL B — LOAD FACTOR

TABLE SIZE	ITEMS HELD	LOAD FACTOR
10	7	**0.7**
13	9	**0.69**
20	5	**0.25**
8	8	**1.0** (full)

λ = items ÷ table size. As λ rises, collisions rise and lookup slows towards O(n); a good hash function keeps λ low, which is why resizing (rehashing into a bigger table) is done when λ passes a threshold.

7 Sixty-second explanations — say these out loud

If you can say each of these from memory without stopping, you can write it. Cover the right column, say it, then check.

PROMPT	WHAT YOU SHOULD BE ABLE TO SAY
Why asymptotic analysis?	Because actual running time depends on hardware, compiler and language. Ignoring constants and looking at the rate of growth as $n \to \infty$ gives an efficiency measure valid on any machine. Three cases: worst, best, average. Three notations: O upper, Θ tight, Ω lower.
Why is binary search faster?	It discards half the remaining data at every comparison, so the work is $\log_2 n$ rather than n — 10 steps instead of 1024 on a 1024-item list. The price is that the data must be sorted first.
Why is hashing O(1)?	The hash function converts the key directly into an array index, so the item is reached in one step instead of being searched for. It degrades only when collisions force chaining or probing, which is why a low load factor matters.
Why is the BST in-order traversal sorted?	In-order visits left, root, right — and by the BST property everything in the left sub-tree is smaller than the root and everything in the right is larger. Applied recursively, the values come out in ascending order.
Stack or queue — how do I choose?	LIFO problems take a stack: recursion and function calls, undo, DFS, expression evaluation. FIFO problems take a queue: CPU scheduling, printer spooling, BFS, buffers — anything where fairness of arrival order matters.
Why does dynamic programming help?	Because the sub-problems overlap. Storing each sub-result once and reusing it turns exponential recomputation into linear work — Fibonacci goes from $O(2^n)$ to O(n). It only applies with overlapping sub-problems and optimal substructure.
What is the time–space tradeoff?	An algorithm can be made faster by spending memory, or made leaner by spending time. A hash table buys O(1) search with O(n) space; merge sort buys a guaranteed O(n log n) with O(n) space; memoization buys speed with a table.
Array or linked list?	Array for random access — O(1) by index — but a fixed size and O(n) insertion in the middle. Linked list for cheap insertion and deletion and a size that grows at run time, but O(n) access because you must walk from the head, plus memory overhead for pointers.
Matrix or list for a graph?	Adjacency matrix for dense graphs: $O(V^2)$ space but O(1) edge lookup. Adjacency list for sparse graphs: O(V + E) space, but checking one edge costs O(degree). Both traversals, BFS and DFS, run in O(V + E) on a list.

8 Rapid-fire recall — cover the right column

PROMPT	ANSWER
Definition of an algorithm	Step-by-step procedure / set of instructions to solve a specific problem
Any five types of algorithm	Brute force, recursive, encryption, backtracking, search, sort, divide & conquer, greedy, DP, randomized
Three cases of running time	Worst, best, average
Notation for the tight bound	Θ (Theta)
Notation for best case	Ω (Omega)
Formal condition for Big O	$0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$
Five complexity classes in order	$O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)$
Complexity of a nested loop	$O(n^2)$
Complexity of halving a problem	$O(\log n)$
Five sorts named in class	Merge, quick, bucket, heap, counting
Merge sort complexity and space	$O(n \log n)$ always; $O(n)$ extra space
Quick sort worst case and when	$O(n^2)$, when the pivot is always the smallest or largest
Two non-comparison sorts	Counting sort, bucket sort
Binary search requirement	The data must be sorted
Sequential search worst case	$O(n)$
Hashing in one word	Lookup
Three components of hashing	Key, hash function, hash table
What a hash function returns	The index of an element in the hash table
Definition of a collision	$h(x) = h(y)$ — two different keys, same hash value
Two collision-handling methods	Separate chaining, open addressing

PROMPT	ANSWER
Load factor formula	Items in the table ÷ size of the table
Four properties of a good hash function	Efficiently computable, uniform distribution, minimizes collision, low load factor
Hash of "efg" in a 7-slot table	$5 + 6 + 7 = 18 \rightarrow 18 \bmod 7 =$ index 4
Three categories of data type	Inbuilt/primitive, derived, complex
Six basic operations	Traversal, searching, sorting, merging, insertion, deletion
Three types of linked list	Simple (singly), complex (doubly), circular
Stack principle and operations	LIFO; PUSH and POP; open at one end
Queue principle and operations	FIFO; Enqueue() and Dequeue(); open at both ends
Pointers a queue needs	Two — front and rear
Three properties of a tree	One root with no parent; one parent per node but many children; nodes joined by edges
Depth vs height	Depth = edges from the root down; height = edges down to the deepest leaf
Height of a leaf	0
Degree of a node	Its number of branches
Full vs perfect binary tree	Full: every node has 0 or 2 children. Perfect: full *and* all leaves on the same level
Perfect tree formulas	$l = 2^h$; $n = 2 \text{^} (h+1) - 1$
Order of post-order traversal	Left → Right → Root
Missing successor / predecessor	-1 / null
Graph definition	$G = (V, E)$ — a set of vertices and a set of edges joining them
Data structure used by BFS / DFS	Queue / stack
Adjacency matrix vs list space	$O(V^2)$ vs $O(V + E)$

9 Spot the error — every statement below is wrong except three

#	STATEMENT	T / F	CORRECTION
1	Binary search works on any array.	**F**	It requires a **sorted** array. On unsorted data it is simply wrong, not just slow.
2	Θ notation describes the worst case only.	**F**	Θ is the **tight** bound — upper *and* lower. Big O is the worst-case upper bound.
3	A queue is open at one end.	**F**	A queue is open at **both** ends. The **stack** is open at one end only.
4	The in-order traversal of any binary tree is sorted.	**F**	Only of a binary **search** tree. An ordinary binary tree has no ordering property.
5	The height of a leaf node is 1.	**F**	It is **0** — there are no edges below a leaf.
6	Quick sort's worst case is O(n log n).	**F**	Its worst case is **$O(n^2)$**; O(n log n) is its average. Merge sort is the one guaranteed O(n log n).
7	Hash table lookup is O(1) on average.	**T**	Correct — with a good hash function and a low load factor.
8	Merge sort sorts in place.	**F**	It needs **O(n)** extra space for merging. Heap sort is the O(1)-space O(n log n) sort.
9	DFS uses a queue.	**F**	DFS uses a **stack** (or recursion); **BFS** uses the queue.
10	A perfect binary tree of height h has 2^h leaves.	**T**	Correct — and 2 ^ (h+1) − 1 nodes in total.
11	Load factor = table size ÷ number of items.	**F**	It is the other way round: **items ÷ table size**.
12	Every full binary tree is a complete binary tree.	**F**	Not necessarily — full only requires 0 or 2 children; complete additionally requires every level filled with leaves leaning left.
13	Stack memory is freed by the programmer.	**F**	Stack frames are freed **automatically** on return. The **heap** is freed manually or by a garbage collector.
14	Counting sort is a comparison sort.	**F**	It is **not** — it counts occurrences instead of comparing, which is how it beats the O(n log n) bound.
15	A tree is a connected acyclic graph.	**T**	Correct — a useful line to quote in a graph question.

10 Walk-in sheet — the last ten minutes

THE EIGHT FACTS MOST LIKELY TO BE TESTED

1. **O** upper/worst · **Θ** tight/both · **Ω** lower/best · bound holds for $\mathbf{n \geq n_0}$
2. $\mathbf{O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)}$
3. Hashing = **lookup** · key → hash function → hash table · **O(1)** · collision = $\mathbf{h(x) = h(y)}$ · chaining or open addressing · λ = items ÷ size
4. Stack **LIFO**, one end, PUSH/POP · Queue **FIFO**, both ends, Enqueue/Dequeue
5. **In** = L-Root-R · **Pre** = Root-L-R · **Post** = L-R-Root · BST in-order is **sorted**
6. Depth = down from the root · Height = up from the deepest leaf · leaf height **0**
7. Missing successor → $\mathbf{-1}$ · missing predecessor → **null**
8. Binary search **O(log n)**, needs **sorted** data · sequential **O(n)**, needs nothing

THREE THINGS TO DO IN THE FIRST FIVE MINUTES

1. Read every question, then start with the one you know best — bank the easy marks while you are fresh.
2. For each question, jot the **skeleton** in the margin before writing: definition → list → example → diagram.
3. Draw any requested diagram **first**. It is the cheapest mark on the paper and the one people run out of time for.

IF YOUR MIND GOES BLANK

Write the **definition** of the term in the question, then the **list** that belongs to it, then an **example**. Every answer on this paper is built from those three moves, and each one scores on its own.